

# Manual Book

## End-to-End Pipeline Mask R-CNN + MobileNetV3-FPN untuk Deteksi dan Segmentasi Lubang Jalan pada Jetson Orin Nano

Dokumen ini menjelaskan langkah dari awal sampai akhir untuk menjalankan pipeline training di laptop, evaluasi model, export ONNX, deployment ke Jetson Orin Nano, konversi TensorRT FP16, benchmark, dan pembuatan laporan akhir.

**Catatan:** training dilakukan di laptop/PC. Jetson Orin Nano digunakan untuk inferensi ONNX Runtime dan TensorRT.

## 1. Gambaran Umum Pipeline

Pipeline terdiri dari tahap dataset COCO, training Mask R-CNN + MobileNetV3-Large + Feature Pyramid Network (FPN), evaluasi kuantitatif, export Open Neural Network Exchange (ONNX), validasi ONNX Runtime, package deployment Jetson Orin Nano, konversi TensorRT FP16, inferensi video/kamera, benchmark, dan laporan akhir.

```
Laptop/PC -> Training PyTorch -> best_model.pth
best_model.pth -> Export ONNX -> maskrcnn_mobilenetv3_fpn_416.onnx
ONNX -> Jetson Orin Nano -> ONNX Runtime inference
ONNX -> TensorRT FP16 -> .engine -> TensorRT inference
Benchmark -> Final deployment report
```

## 2. Struktur Folder Project

Setelah ZIP diekstrak, struktur folder harus dipertahankan agar notebook dapat membaca konfigurasi YAML dan dataset dengan benar.

```
maskrcnn_jetson_orin_final_v2/
├── configs/
│   └── config_maskrcnn_mobilenetv3_fpn.yaml
├── notebooks/
│   └── maskrcnn_mobilenetv3_fpn_jetson_orin_end_to_end_v2.ipynb
├── data/
│   ├── images/train/
│   ├── images/val/
│   ├── images/test/
│   ├── annotations/train.json
│   ├── annotations/val.json
│   ├── annotations/test.json
│   └── videos/pothole_test.mp4
├── checkpoints/
├── outputs/
└── docs/
```

## 3. Persiapan Environment Laptop

Gunakan Python 3.10 atau versi yang kompatibel dengan PyTorch dan TorchVision. Untuk Windows, disarankan memakai Anaconda dan Visual Studio Code dengan ekstensi Jupyter.

```
conda create -n pothole_maskrcnn python=3.10 -y
conda activate pothole_maskrcnn
pip install -r requirements.txt
python -m ipykernel install --user --name pothole_maskrcnn --display-name "pothole_maskrcnn"
```

## 4. Format Dataset COCO

Dataset harus menggunakan format COCO instance segmentation. Setiap gambar training, validation, dan test harus memiliki anotasi bbox dan segmentation polygon/RLE. Untuk satu kelas lubang jalan, semua kategori COCO dapat dipetakan ke label pothole=1.

```
data/images/train/    -> gambar train
data/images/val/      -> gambar validasi
data/images/test/     -> gambar test
data/annotations/train.json
data/annotations/val.json
data/annotations/test.json
```

## 5. Konfigurasi YAML

Semua hyperparameter utama dibaca dari file YAML. Jika YAML diubah, jalankan ulang notebook mulai Cell 1 agar variabel notebook ikut berubah.

```
dataset:
  input_size: 416

training:
  epochs: 50
  batch_size: 2

inference:
```

```
score_threshold: 0.5
mask_threshold: 0.5

video_inference:
  process_every_n_frames: 1
  max_frames: 100
```

## 6. Urutan Cell Notebook

Jalankan notebook secara berurutan. Jangan langsung lompat ke export ONNX sebelum training menghasilkan `best_model.pth`.

```
Cell 1-3 : setup library, YAML, folder
Cell 4-5 : dataset COCO dan DataLoader
Cell 6   : Mask R-CNN + MobileNetV3-FPN
Cell 7   : training loop dan checkpoint
Cell 8-12 : inferensi gambar, batch, video
Cell 13-15 : evaluasi, COCO mAP, profiling
Cell 16-17 : ONNX export dan validasi
Cell 18-23 : deployment Jetson Orin Nano sampai laporan akhir
```

## 7. Training Model di Laptop

Untuk uji awal gunakan epoch kecil terlebih dahulu. Jika proses berjalan tanpa error dan checkpoint tersimpan, naikan epoch ke nilai penelitian.

```
# YAML awal uji cepat
training:
  epochs: 5
  batch_size: 1

# training serius
training:
  epochs: 50
  batch_size: 2
```

## 8. Inferensi Gambar dan Video

Setelah `best_model.pth` dimuat, jalankan inferensi satu gambar, batch test, lalu video. Video default bernama `pothole_test.mp4` di folder `data/videos`.

```
data/videos/pothole_test.mp4
outputs/predictions/pothole_test_prediction.mp4
outputs/predictions/video_inference_summary.csv
```

## 9. Evaluasi Kuantitatif dan COCO mAP

Cell 13 menghitung precision, recall, F1-score, box IoU, mask IoU, latency, dan FPS. Cell 14 menghitung COCO bbox mAP dan segmentation mAP. Untuk publikasi, COCO mAP lebih kuat dibanding visualisasi saja.

```
outputs/metrics/test_evaluation_summary.json
outputs/metrics/coco_map_summary.json
outputs/metrics/coco_bbox_predictions.json
outputs/metrics/coco_segm_predictions.json
```

## 10. Profiling Model

Cell 15 menghitung ukuran model, parameter, latency, FPS, dan estimasi kelayakan Jetson Orin Nano. Profiling laptop bukan pengganti benchmark di Jetson, tetapi memberi indikasi awal.

```
outputs/metrics/edge_profiling_summary.json
outputs/metrics/latency_distribution_model_only.png
```

## 11. Export ONNX

Cell 16 mengekspor model PyTorch ke ONNX. Untuk Jetson Orin Nano, disarankan membuat model 416 dan opsional 320 sebagai pembanding. File ONNX harus divalidasi sebelum TensorRT.

```
outputs/onnx/maskrcnn_mobilenetv3_fpn_416.onnx
outputs/metrics/onnx_export_summary.json
```

## 12. Validasi ONNX Runtime di Laptop

Cell 17 memastikan file ONNX dapat dijalankan di ONNX Runtime. Jika gagal di laptop, jangan lanjut ke TensorRT. Perbaiki export ONNX terlebih dahulu.

```
pip install onnxruntime onnx
# atau jika GPU laptop kompatibel
pip install onnxruntime-gpu
```

## 13. Package Deployment Jetson Orin Nano

Cell 18 membuat folder deployment berisi ONNX, YAML, scripts, video, requirements, dan README. Folder ini yang dipindahkan ke Jetson Orin Nano.

```
outputs/jetson_orin_deployment/
■■■ models/
■■■ configs/
■■■ scripts/
■■■ videos/
■■■ outputs/
■■■ requirements_jetson_orin.txt
```

## 14. Menjalankan di Jetson Orin Nano

Di Jetson, aktifkan mode performa tinggi sebelum benchmark. TensorRT engine dibuat langsung di Jetson, bukan di laptop.

```
sudo nvpmodel -m 0
sudo jetson_clocks
sudo -H pip3 install -U jetson-stats
jtop
```

## 15. ONNX Runtime Inference di Jetson

Cell 19 membuat script ONNX video/kamera. Jalankan ONNX inference sebagai baseline sebelum TensorRT.

```
chmod +x run_onnx_video_inference.sh
./run_onnx_video_inference.sh

chmod +x run_onnx_camera_inference.sh
./run_onnx_camera_inference.sh
```

## 16. Konversi TensorRT FP16

Cell 20 membuat script konversi ONNX ke TensorRT FP16. Jika FP16 gagal, script mencoba fallback FP32. Mask R-CNN dapat gagal dikonversi karena operator kompleks seperti RoIAlign atau Non-Maximum Suppression (NMS).

```
chmod +x run_tensorrt_conversion.sh
./run_tensorrt_conversion.sh

# output jika berhasil
models/maskrcnn_mobilenetv3_fpn_416_fp16.engine
```

## 17. TensorRT Engine Inference

Cell 21 membuat script inferensi TensorRT engine. Script ini hanya berjalan jika file .engine sudah berhasil dibuat.

```
chmod +x run_tensorrt_video_inference.sh
./run_tensorrt_video_inference.sh

chmod +x run_tensorrt_camera_inference.sh
```

```
./run_tensorrt_camera_inference.sh
```

## 18. Benchmark ONNX Runtime vs TensorRT

Cell 22 membandingkan FPS dan latency ONNX Runtime vs TensorRT. Hasil penting adalah `model_fps_speedup` dan `e2e_fps_speedup`.

```
chmod +x run_benchmark_onnx_vs_tensorrt.sh
./run_benchmark_onnx_vs_tensorrt.sh

outputs/benchmark_onnx_vs_tensorrt/benchmark_onnx_vs_tensorrt_summary.csv
```

## 19. Laporan Akhir Deployment

Cell 23 membuat laporan akhir deployment dalam Markdown, JSON, dan CSV. Laporan ini dapat digunakan untuk dokumentasi riset atau lampiran eksperimen.

```
chmod +x run_generate_deployment_report.sh
./run_generate_deployment_report.sh

outputs/final_deployment_report/jetson_orin_deployment_report.md
```

## 20. Error Umum dan Solusi - Dataset

Error dataset biasanya terjadi karena path salah, file JSON tidak ditemukan, nama file pada COCO JSON tidak cocok dengan gambar, atau anotasi tidak memiliki segmentation.

```
Error: FileNotFoundError train.json
Solusi: pastikan data/annotations/train.json ada.

Error: gambar tidak ditemukan
Solusi: cek field file_name pada COCO JSON dan nama file asli.

Error: annToMask gagal
Solusi: pastikan anotasi memiliki segmentation polygon/RLE valid.
```

## 21. Error Umum dan Solusi - Training

Training Mask R-CNN membutuhkan memori GPU cukup besar. Jika CUDA out of memory, turunkan batch size dan input size.

```
Error: CUDA out of memory
Solusi:
    training.batch_size: 1
    dataset.input_size: 320
    rpn_post_nms_top_n_train: 300

Error: loss NaN
Solusi:
    cek bbox invalid, mask kosong, anotasi rusak
    turunkan learning_rate menjadi 0.00005
```

## 22. Error Umum dan Solusi - OpenCV

OpenCV dapat gagal karena dependency Windows atau environment Python salah.

```
Error: ImportError DLL load failed while importing cv2
Solusi:
    pip uninstall opencv-python -y
    pip install opencv-python
    pastikan kernel Jupyter memakai environment yang benar
```

## 23. Error Umum dan Solusi - ONNX

ONNX export Mask R-CNN lebih rumit dibanding model klasifikasi. Gunakan input size tetap dan batch size satu.

```
Error: ONNX export failed
```

Solusi:  
inference\_model.eval()  
dummy\_input = [torch.randn(3, INPUT\_SIZE, INPUT\_SIZE).to(device)]  
coba opset 11 atau 12  
coba export di CPU

## 24. Error Umum dan Solusi - TensorRT

TensorRT bisa gagal mengkonversi Mask R-CNN karena operator yang tidak didukung. Ini bukan berarti training gagal.

Error: Unsupported ONNX operator  
Solusi:  
coba FP32  
coba input\_size 320  
gunakan ONNX Runtime sebagai baseline  
jika wajib real-time, bandingkan dengan YOLO segmentation

Error: trtexec not found  
Solusi:  
find /usr -name trtexec 2>/dev/null  
gunakan /usr/src/tensorrt/bin/trtexec

## 25. Error Umum dan Solusi - Kamera Jetson

Kamera mungkin tidak berada di index 0 atau membutuhkan pipeline GStreamer khusus.

Error: Camera index 0 not available  
Solusi:  
coba --source 1  
cek kamera dengan: v4l2-ctl --list-devices  
untuk CSI camera, gunakan pipeline GStreamer khusus

## 26. Rekomendasi Eksperimen Penelitian

Untuk laporan akademik, jangan hanya menampilkan gambar prediksi. Sertakan metrik COCO mAP, precision, recall, F1-score, latency, FPS, ukuran model, dan benchmark ONNX vs TensorRT.

Tabel minimum:

1. COCO bbox mAP dan mask mAP
2. Precision, recall, F1-score
3. Model size dan parameter count
4. PyTorch vs ONNX vs TensorRT FPS
5. Input size 320 vs 416
6. Kelemahan dan batasan deployment

## Lampiran: Checklist End-to-End

| No | Item                                     | Status |
|----|------------------------------------------|--------|
| 1  | Dataset COCO train/val/test tersedia     | []     |
| 2  | YAML sudah diubah sesuai eksperimen      | []     |
| 3  | Cell 1-7 training berhasil               | []     |
| 4  | best_model.pth tersimpan                 | []     |
| 5  | Inferensi gambar dan video berhasil      | []     |
| 6  | Evaluasi test set dan COCO mAP selesai   | []     |
| 7  | ONNX export berhasil                     | []     |
| 8  | ONNX Runtime valid di laptop             | []     |
| 9  | Deployment package dipindahkan ke Jetson | []     |
| 10 | TensorRT conversion diuji di Jetson      | []     |
| 11 | Benchmark ONNX vs TensorRT selesai       | []     |
| 12 | Final deployment report dibuat           | []     |